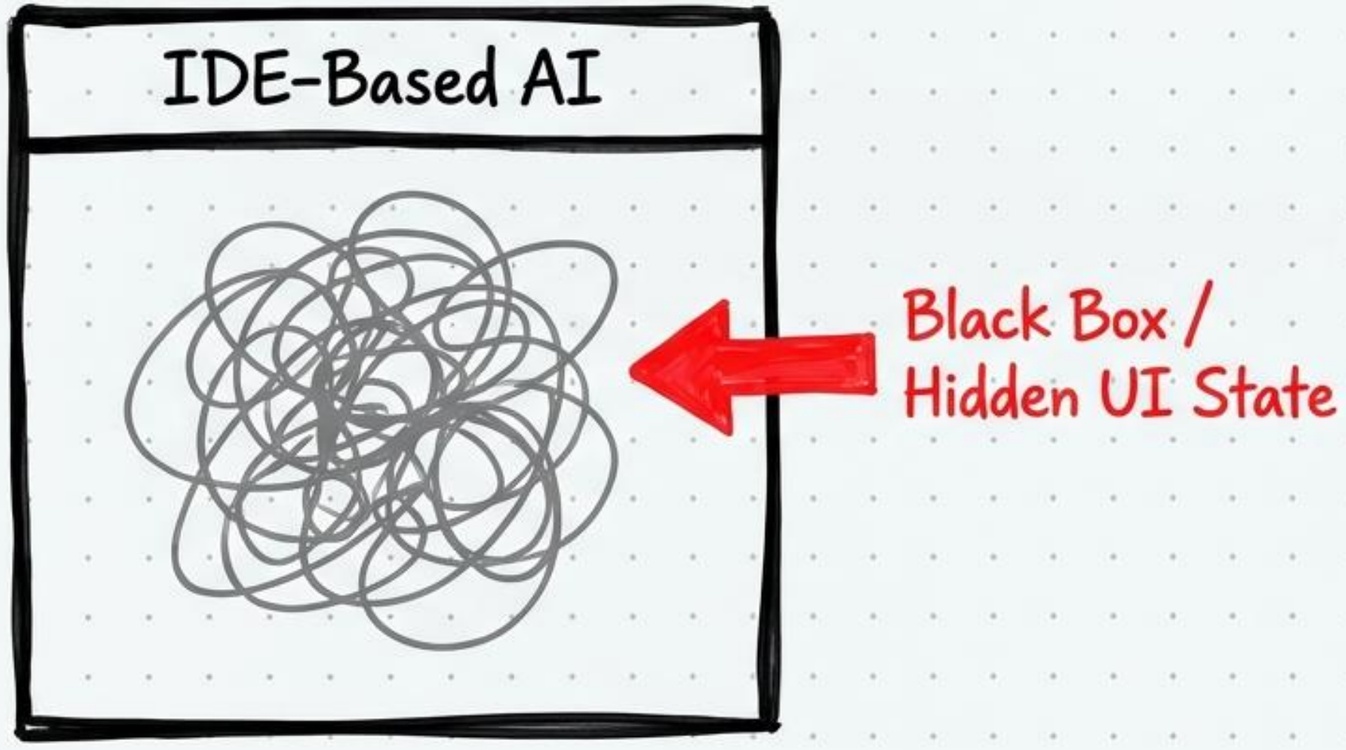
Claude Code: The Definitive Runbook

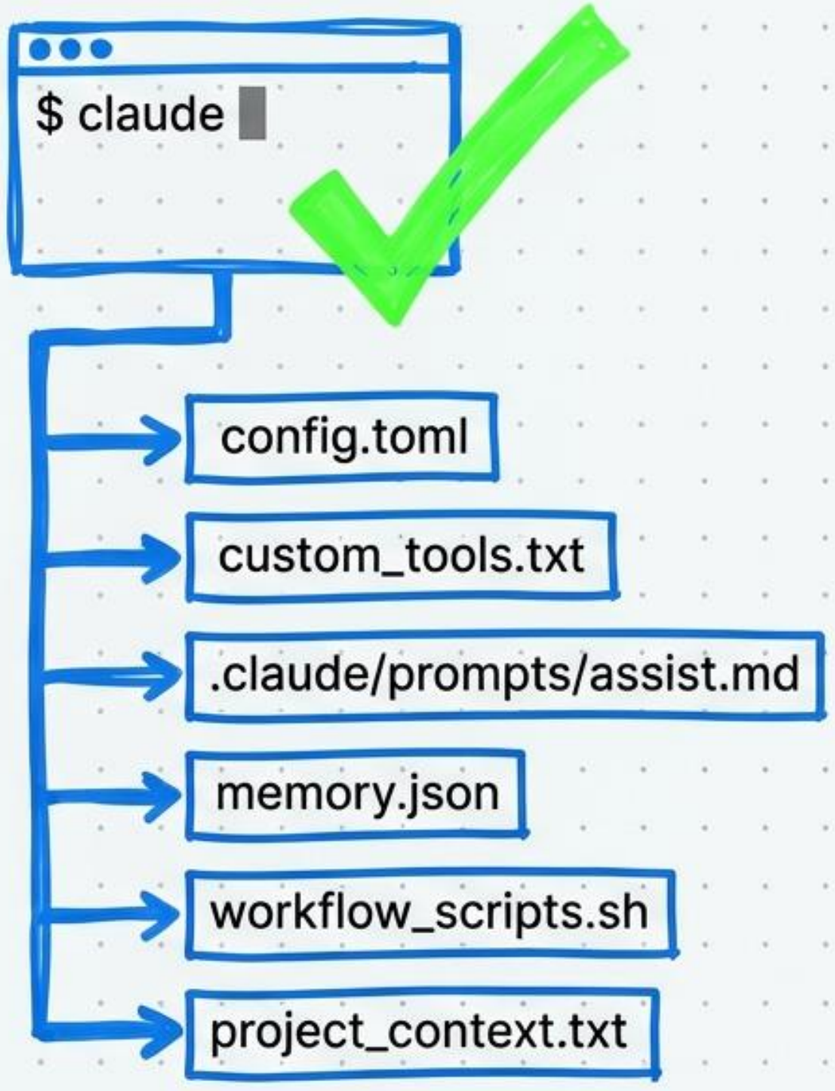
A Principal Engineer's guide to the 5-layer, file-native customization model.

v1.0 | Target: Local Dev Workflows

The Philosophy: File-First, Code-Native

- Claude Code operates from the terminal, not a hidden GUI panel.
- Every customization is a plain text file.
- Everything is tracked in version control. Treat your AI teammate like code.

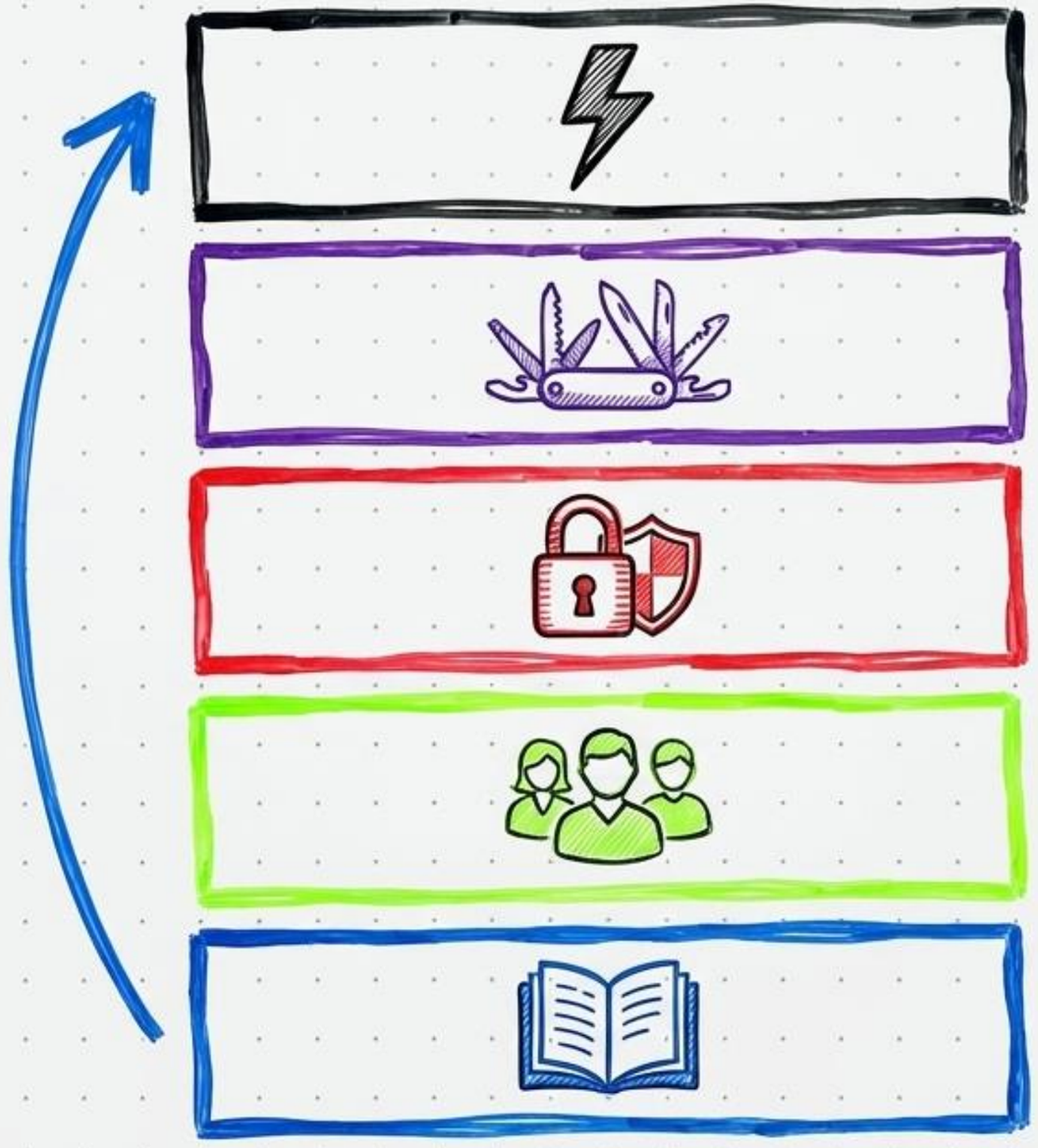




Transparency & Control.
Your AI is part of your codebase.

NotebookLM

The 5-Layer Customization Architecture



Slash Commands

Walkie-Talkie Shortcuts (Repeatable tasks)

MCP Servers

The Toolbelt (External access)

Layer 3: Hooks

Security Checkpoints (Is it safe?)

Layer 2: Sub-Agents

Specialized Teammates (Who does the work)

Layer 1: Memory

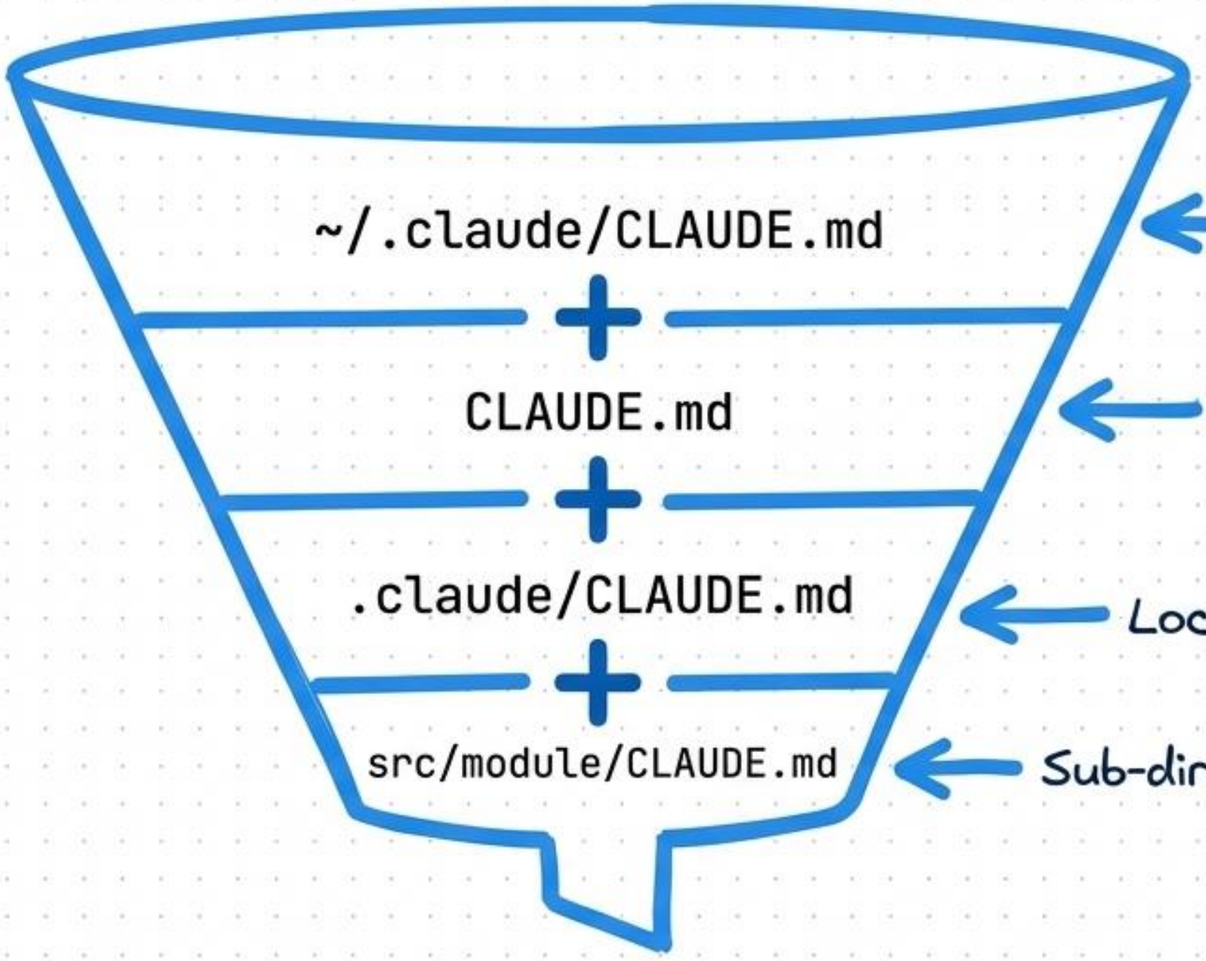
Employee Handbook (Who you are)

Cooperative, layered architecture. Do not put everything in one layer!

NotebookLM

Memory System: The Precedence Funnel

Slash Commands
MCP Servers
Layer 3: Hooks
Layer 2: Sub-Agents
📖 Layer 1: Memory



User Global (100-300 tokens)
- Always-on everywhere

Project Root (200-800 tokens)
- Core stack & conventions

Local - Uncommitted machine overrides

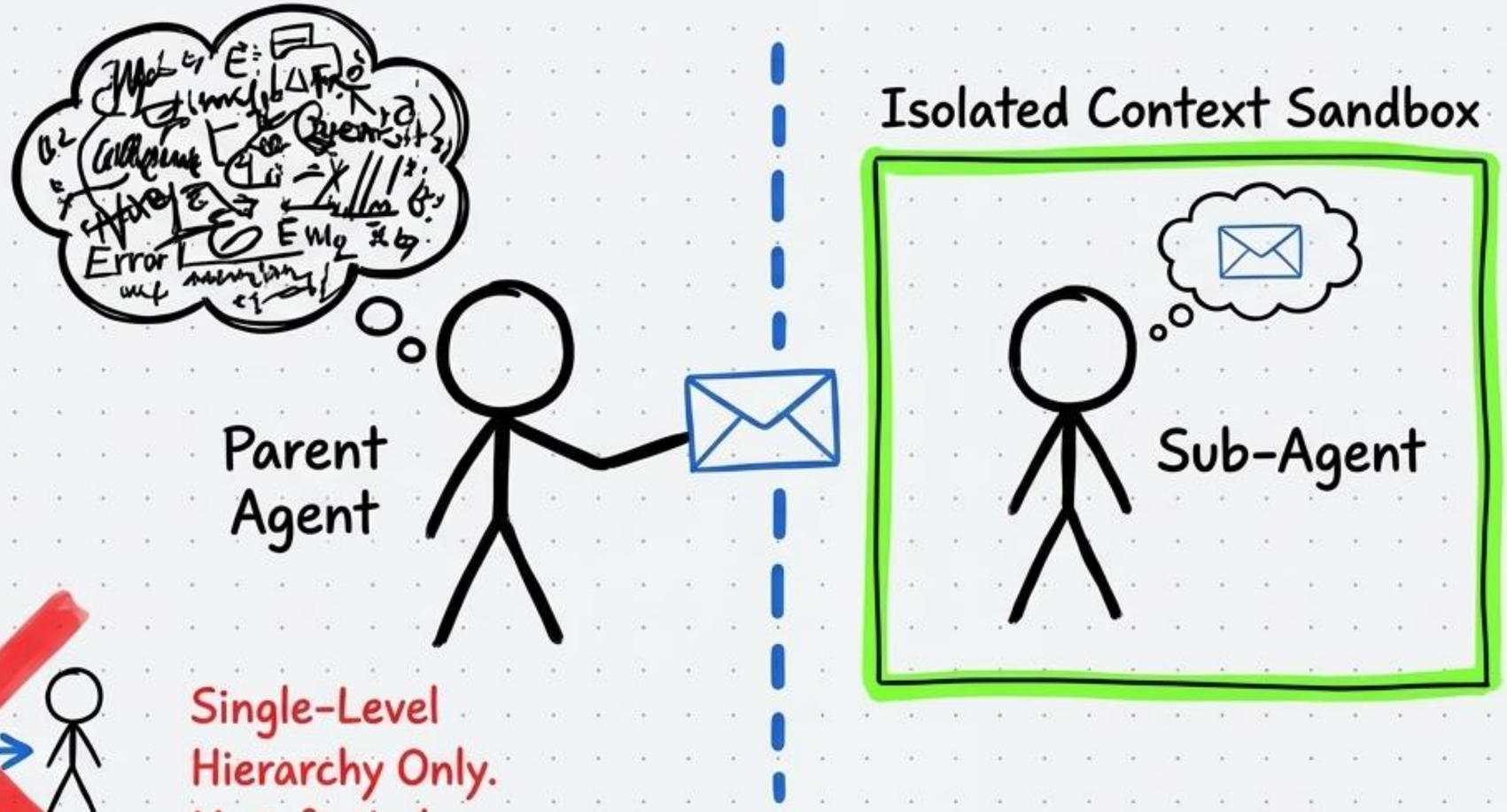
Sub-directory - Scoped to the subtree

Redundant rules = context bloat. Never contradict layers.

NotebookLM

Sub-Agents: The Power of Isolated Context

Sub-agents do not inherit parent history. This guarantees security, token efficiency, and parallel execution.



NotebookLM

Sub-Agents: Implementation Rules

- Slash Commands
- MCP Servers
- Layer 3: Hooks
- Layer 2: Sub-Agents
- Layer 1: Memory

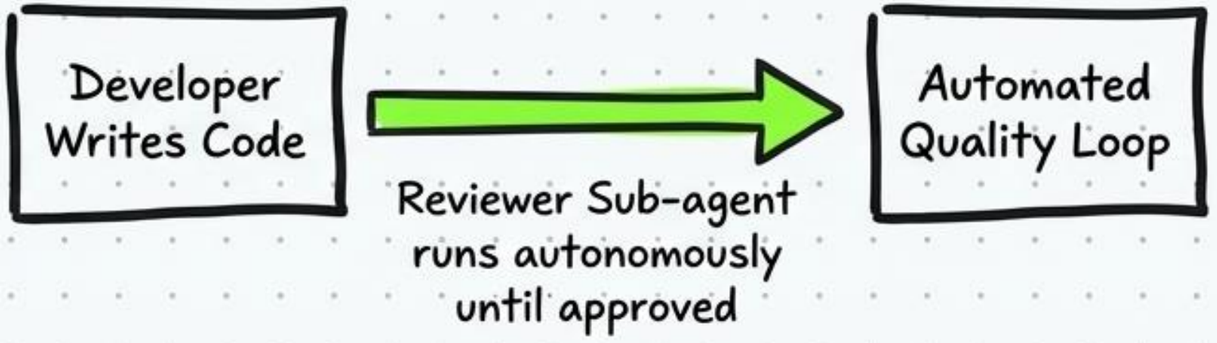
Markdown + YAML

```
name: code-reviewer
description: ...
tools: [Read]
```

CRITICAL: The description is the trigger! Claude reads this to decide when to invoke the agent autonomously. Be explicit.

Storage Location

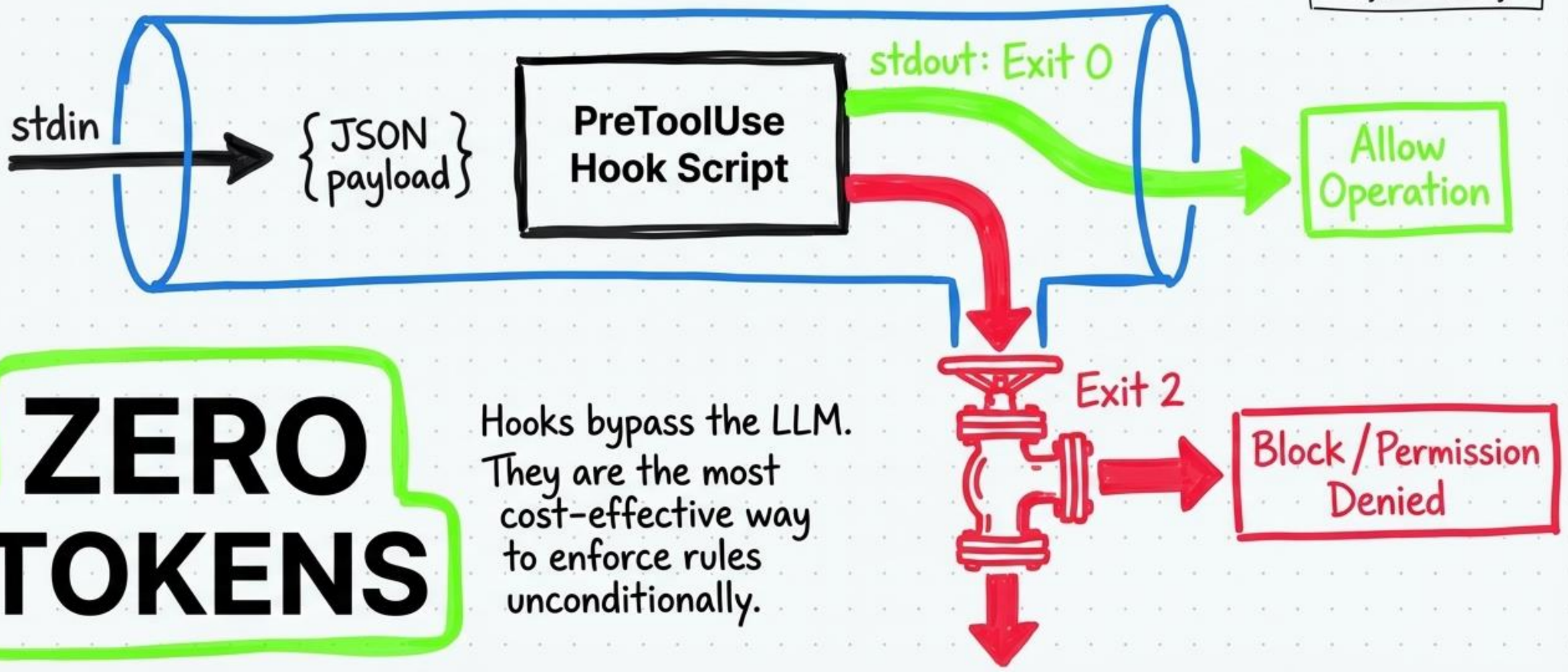
- Global: ~/.claude/agents/*.md
- Project: .claude/agents/*.md



NotebookLM

Hooks: Deterministic Enforcement

Slash Commands
MCP Servers
Layer 3: Hooks
Layer 2: Sub-Agents
Layer 1: Memory

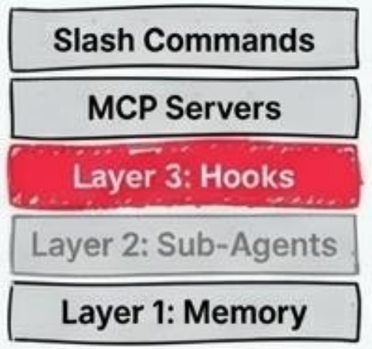


ZERO TOKENS

Hooks bypass the LLM. They are the most cost-effective way to enforce rules unconditionally.

NotebookLM

Practical Application of Hooks



Event Type	Trigger	Use Case
PreToolUse	Before tool call	Block dangerous shell commands (e.g., `rm -rf`)
PostToolUse	After tool call	Auto-run linter/formatter on save
SubagentStop	Sub-agent finishes	Aggregate results / log output

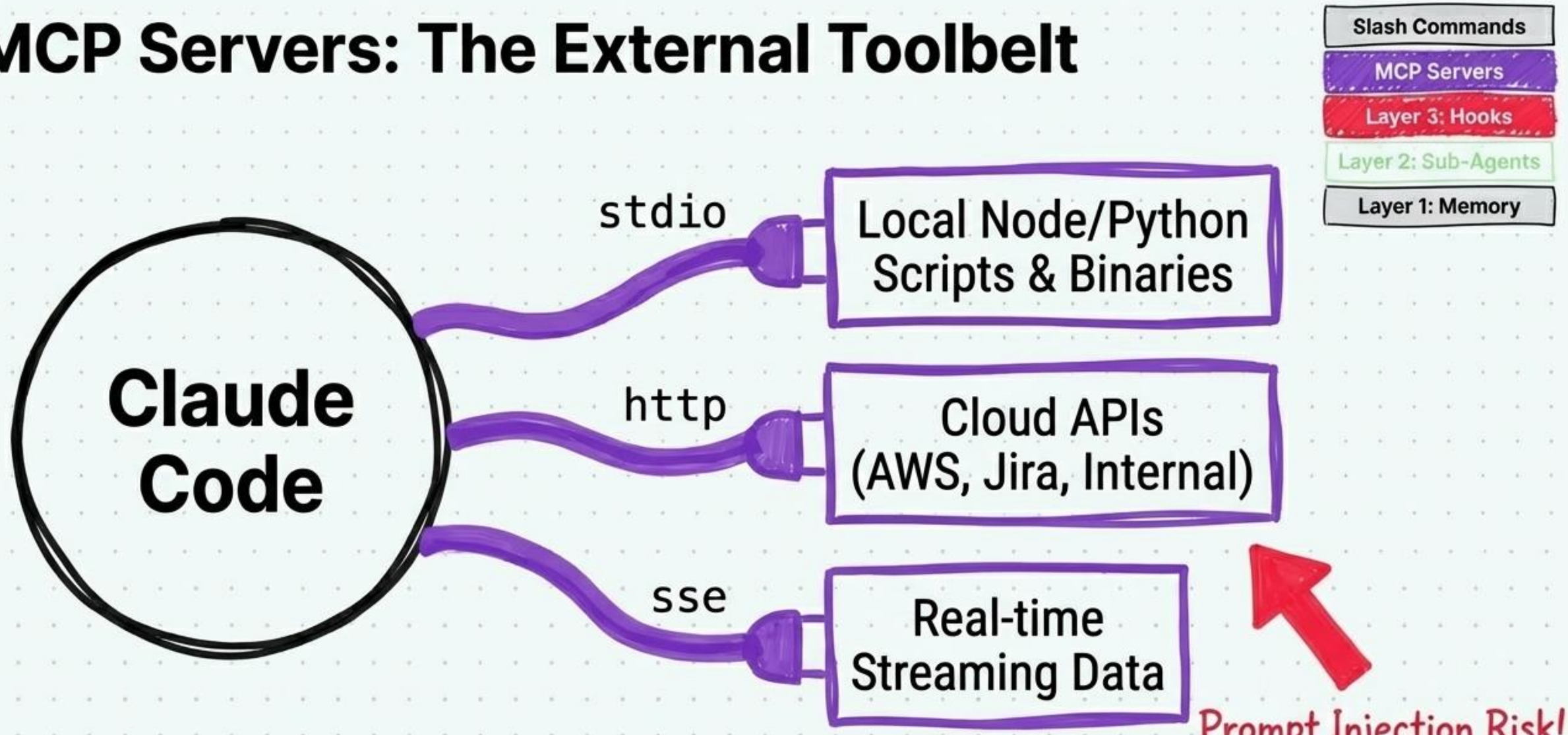
.claude/settings.json

```
"hooks": {
   "PreToolUse": "node check-perms.js"
}
```

Never use a CLAUDE.md instruction (“Always lint after saving”) for something a Hook can do deterministically.

NotebookLM

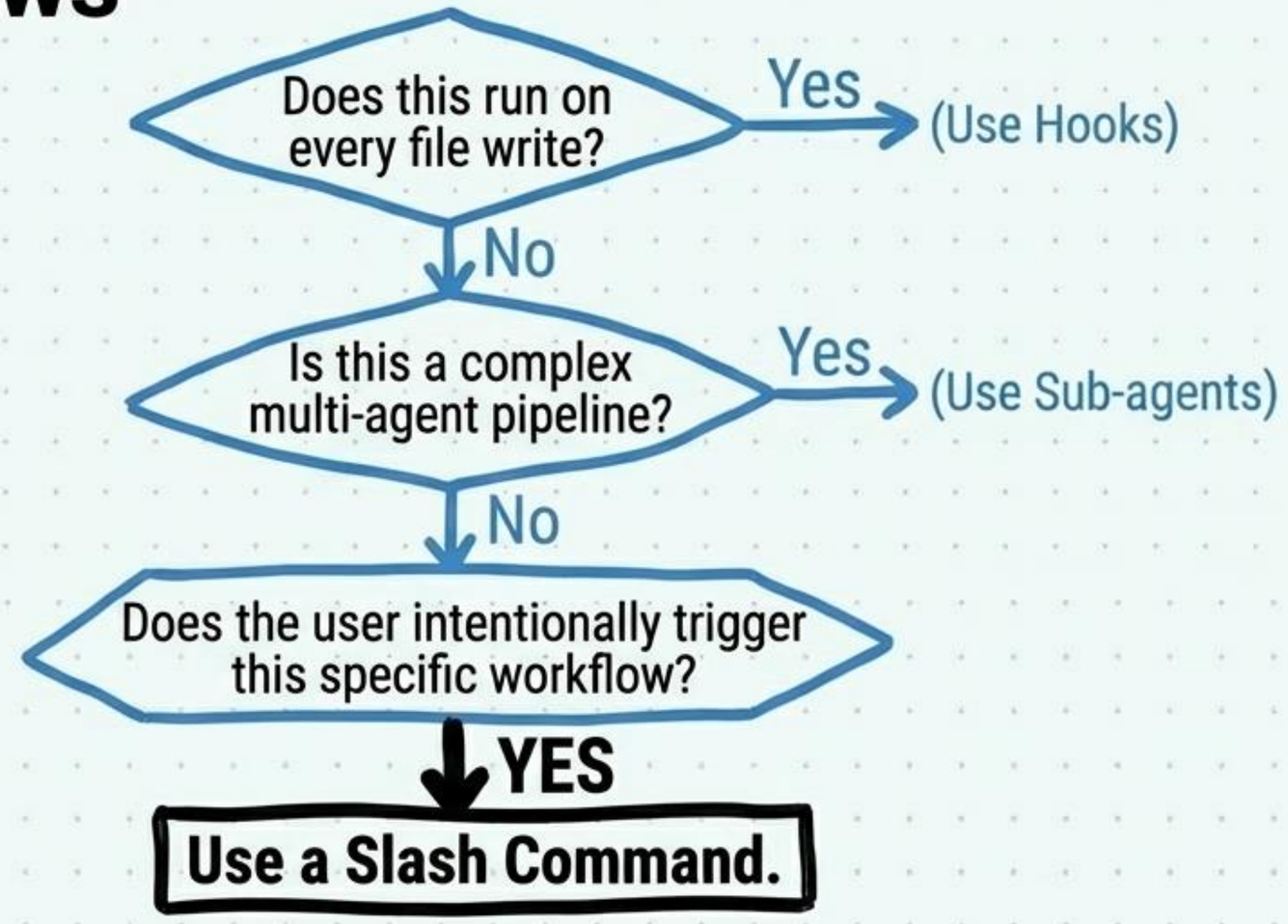
MCP Servers: The External Toolbelt

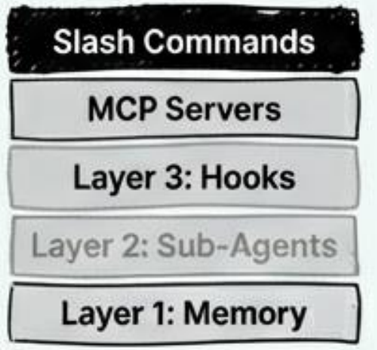


Configured via `mcpServers` key in `.claude/settings.json`.

NotebookLM

Slash Commands: User-Triggered Workflows





`/project:<name>`
`$ARGUMENTS`

Stored as Markdown in `.claude/commands/`. The ultimate reusable prompt macro for on-demand task execution.

NotebookLM

The Diagnostic Matrix: Where Does the Workflow Belong?

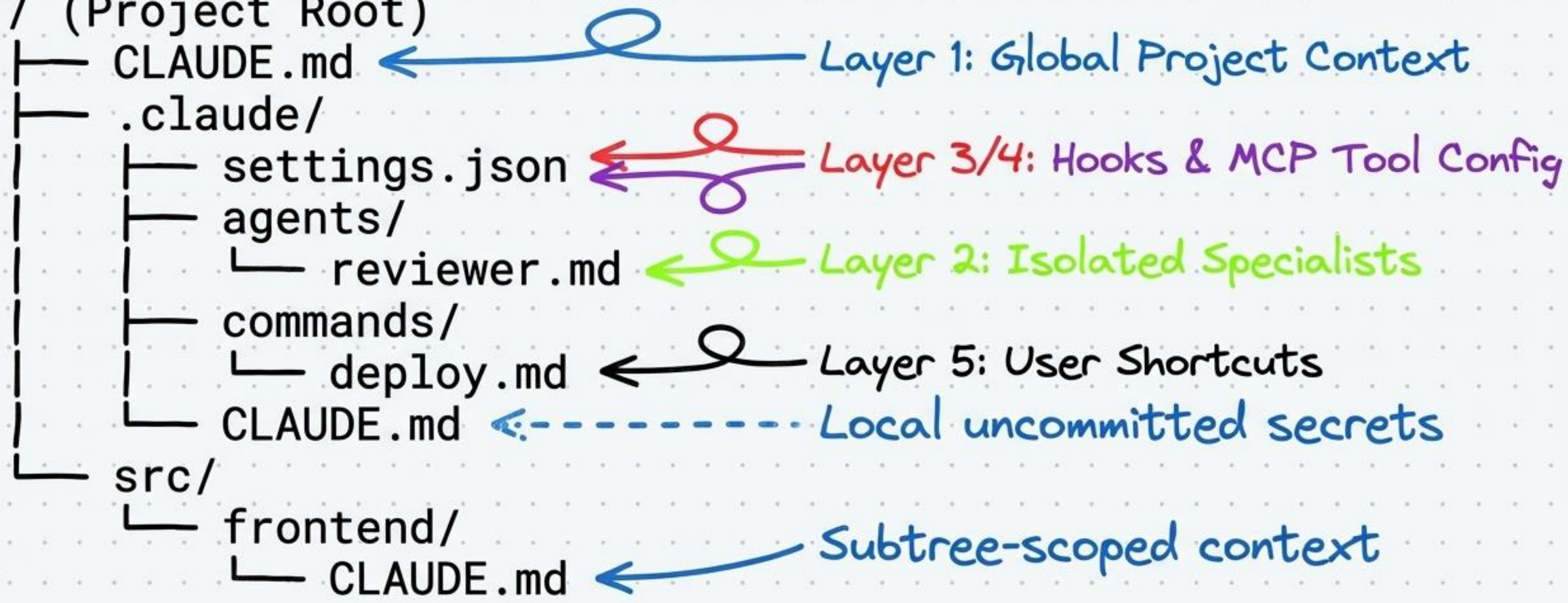
Stop putting 15-step debugging workflows in your root CLAUDE.md!

Reserve this strictly for 'Always-On' context.

	Memory	Hooks	Sub-agents	Slash Cmds	MCP Servers
Global Standards & Conventions	✓				
Lint-on-save / Block specific commands		✓			
Automated Parallel Code Review			✓		
User-Triggered Database Migration				✓	
Accessing Internal Jira Tickets					✓

NotebookLM

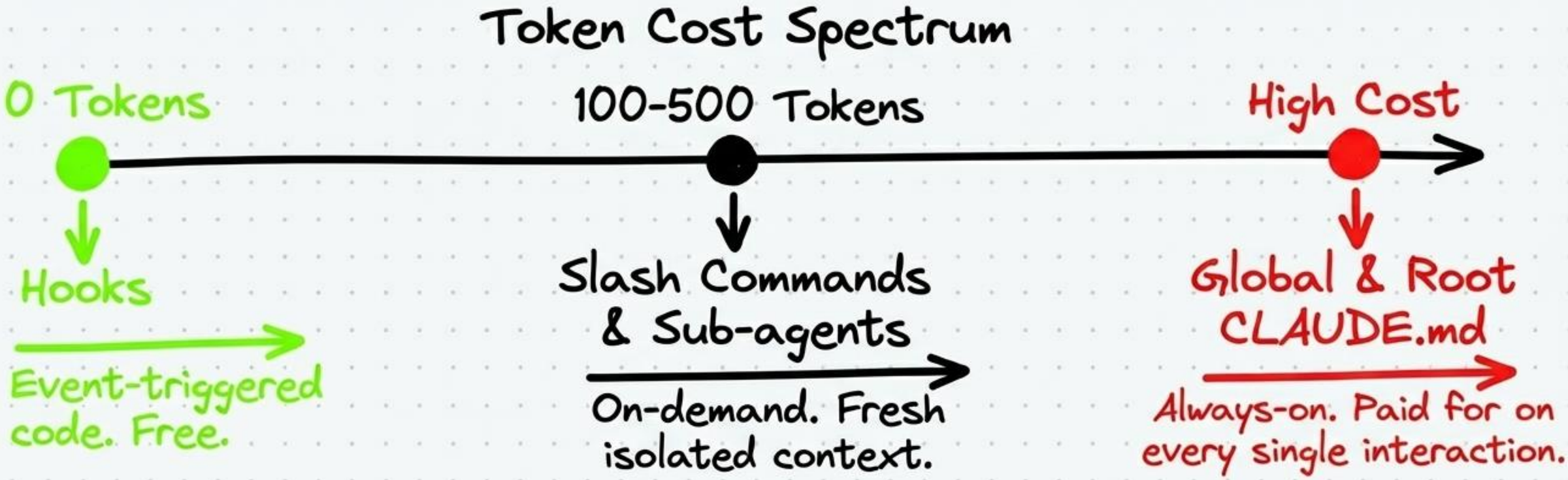
Synthesis: The Complete Project Structure



The Complete Blueprint. Everything in its right place, checked into version control, cooperating seamlessly.

NotebookLM

Engineering Constraints: Token Economics



The Golden Rule

Never **always-on** what can be on-demand. Sub-agents are cheaper than context accumulation.

NotebookLM

Engineering Constraints: Security Posture

Default Deny (.claude/settings.json)

```
!Bash(*)
```

Explicitly enumerate allowed commands.
Deny all shell access by default.

Least Privilege (Sub-agents)

```
disallowedTools: [ExecuteCommand]
```

Reviewers don't need write access.
Restrict tool sets per agent.

Container Isolation

Risky agents (builders, migrators) must run inside containers. Never use `bypassPermissions` in production.

NotebookLM

Architectural Teardown: Claude Code vs. GitHub Copilot

	Claude Code	GitHub Copilot
Environment	Terminal-first	IDE-first
Sub-agent Definition	Markdown + YAML (.claude/agents/)	.agent.md (.github/agents/)
Hooks	Stdin/Stdout Exit Code 2 (Strict Block)	UI permissions toggle
Token Efficiency	Hooks = 0 tokens; Agents strictly isolated	Instructions are always-on

Build for the terminal. Version control your AI. Architect for reliability.

NotebookLM